

Network Programming Project 2 - Remote Working Ground (rwg) Server

NP TA

Deadline: Sunday, 2026/5/3 23:59

1 Introduction

In this project, you are asked to design 2 kinds of servers:

1. Design a server of the chat-like systems, called remote working systems (rwg). In this system, users can communicate with other users. You need to use the **single-process concurrent** paradigm to design this server.
2. Design the rwg server using the **concurrent connection-oriented** paradigm, which means the server will create new process for each client. You need to use **shared memory** and **FIFO** for inter-process communication.

These two servers must support all functions in project 1.

2 Scenario of both servers

2.1 Introduction of Requirements

The servers should support the following features:

1. All commands in project 1
2. Broadcast message of login/logout information.
3. New built-in commands:
 - who: show information of all users.
 - tell: send a message to another user.
 - yell: send a message to all users.
 - name: change your name.
4. Pipe between different users. Broadcast message whenever a user pipe is used.

More details will be defined in Section 3.

2.2 Scenario

The following is a scenario of using the rwg system.

Assume your server is running on nplinux1 and listening at port 7001.

```
bash$ telnet nplinux1.nycu.edu.tw 7001
*****
** Welcome to the information server. **
*****
*** User '(no name)' entered from 140.113.215.62:1201. *** # Welcome message
% who
<ID>    <fd>    <nickname> <IP:port>  <indicate me>
1   3   (no name)  140.113.215.62:1201 <-me
% name Pikachu
*** User from 140.113.215.62:1201 is named 'Pikachu'. ***
% ls
bin test.html
% *** User '(no name)' entered from 140.113.215.63:1013. *** # User 2 logins
who
<ID>    <fd>    <nickname> <IP:port>  <indicate me>
```

```

1 3 Pikachu 140.113.215.62:1201 <-me
2 8 (no name) 140.113.215.63:1013
% *** User from 140.113.215.63:1013 is named 'Snorlax'. *** # User 2 inputs 'name Snorlax'
who
<ID> <fd> <nickname> <IP:port> <indicate me>
1 3 Pikachu 140.113.215.62:1201 <-me
2 8 Snorlax 140.113.215.63:1013
% *** User '(no name)' entered from 140.113.215.64:1302. *** # User 3 logs in
who
<ID> <fd> <nickname> <IP:port> <indicate me>
1 3 Pikachu 140.113.215.62:1201 <-me
2 8 Snorlax 140.113.215.63:1013
3 10 (no name) 140.113.215.64:1302
% yell Who knows how to do project 2? help me plz!
*** Pikachu yelled ***: Who knows how to do project 2? help me plz!
% *** (no name) yelled ***: Sorry, I don't know. :-( # User 3 yells
*** Snorlax yelled ***: I know! It's too easy! # User 2 yells
tell 2 Plz help me, my friends!
% *** Snorlax told you ***: Yeah! Let me show you how to send files to you! # User 2 tells to User 1
*** Snorlax (#2) just piped 'cat test.html >1' to Pikachu (#1) *** # Broadcast message of user pipe
*** Snorlax told you ***: You can use 'cat <2' to show it!
cat <5 # mistyping
*** Error: user #5 does not exist yet. ***
% cat <2 # receive from the user pipe
*** Pikachu (#1) just received from Snorlax (#2) by 'cat <2' ***
<!test.html>
<TITLE>Test<TITLE>
<BODY>This is a <b>test</b> program for rwg.</BODY>
% tell 2 It's works! Great!
% *** Snorlax (#2) just piped 'number test.html >1' to Pikachu (#1) ***
*** Snorlax told you ***: You can receive by your program! Try 'number <2'!
number <2
*** Pikachu (#1) just received from Snorlax (#2) by 'number <2' ***
1 1 <!test.html>
2 2 <TITLE>Test<TITLE>
3 3 <BODY>This is a <b>test</b> program for rwg.</BODY>
% tell 2 Cool! You're genius! Thank you!
% *** Snorlax told you ***: You're welcome!
*** User 'Snorlax' left. ***
exit
bash$

```

Now, let's see what happened to the second user:

```

bash$ telnet nplinux1.nycu.edu.tw 7001 # The server port number
*****
** Welcome to the information server. **
*****
*** User '(no name)' entered from 140.113.215.63:1013. ***
% name Snorlax
*** User from 140.113.215.63:1013 is named 'Snorlax'. ***
% *** User '(no name)' entered from 140.113.215.64:1302. ***
who
<ID> <fd> <nickname> <IP:port> <indicate me>
1 3 Pikachu 140.113.215.62:1201
2 8 Snorlax 140.113.215.63:1013 <-me
3 10 (no name) 140.113.215.64:1302
% *** Pikachu yelled ***: Who knows how to do project 2? help me plz!
*** (no name) yelled ***: Sorry, I don't know. :-(
yell I know! It's too easy!
*** Snorlax yelled ***: I know! It's too easy!
% *** Pikachu told you ***: Plz help me, my friends!
tell 1 Yeah! Let me show you how to send files to you!

```

```
% cat test.html >1 # write to the user pipe
*** Snorlax (#2) just piped 'cat test.html >1' to Pikachu (#1) ***
% tell 1 You can use 'cat <2' to show it!
% *** Pikachu (#1) just received from Snorlax (#2) by 'cat <2' ***
*** Pikachu told you ***: It's works! Great!
number test.html >1
*** Snorlax (#2) just piped 'number test.html >1' to Pikachu (#1) ***
% tell 1 You can receive by your program! Try 'number <2'!
% *** Pikachu (#1) just received from Snorlax (#2) by 'number <2' ***
*** Pikachu told you ***: Cool! You're genius! Thank you!
tell 1 You're welcome!
% exit
bash$
```

3 Spec Details

3.1 Working Directory

```
your_working_directory
|-- bin
|   |-- cat
|   |-- ls
|   |-- noop
|   |-- number
|   |-- removetag
|
|   |-- user_pipe (Will be crated by the test script)
|   |-- (You should create your user pipe FIFO file here)
|
|-- test.html
```

3.2 Format of the Commands

- who:
Show information of all users.

Output Format:

Sort by ID in ascending order.

```
<ID>[Tab]<fd>[Tab]<nickname>[Tab]<IP:port>[Tab]<indicate me>
(1st id)[Tab](1st fd id)[Tab](1st name)[Tab](1st IP:port)([Tab](<-me))
(2nd id)[Tab](2nd fd id)[Tab](2nd name)[Tab](2nd IP:port)([Tab](<-me))
(3rd id)[Tab](3rd fd id)[Tab](3rd name)[Tab](3rd IP:port)([Tab](<-me))
...
```

Example:

```
% who
<ID>    <fd>    <nickname>  <IP:port>    <indicate me>
1   3   IamStudent  140.113.215.62:1201  <-me
2   4   (no name)   140.113.215.63:1013
3   5   student3    140.113.215.62:1201
```

Note that the delimiter of each column is a **Tab** (`\t`), and the first column represents the login user-id. The user's id should be assigned in the range of 1-30.

Your server should always assign the **smallest unused id** to a new user.

Example:

```
<new user login> // server assigns this user id = 1
<new user login> // server assigns this user id = 2
<user 1 logout>
<new user login> // server assigns this user id = 1, not 3
```

- `tell <user id> <message>`:

The user will get the message with following format:

```
*** <sender's name> told you ***: <message>
```

If the receiver of the message doesn't exist, print the following message:

```
*** Error: user #<user id> does not exist yet. ***
```

Example:

```
Assume my name is 'IamStudent'
[terminal of mine]
% tell 3 Hello World.
%

If user 3 exists
[terminal of user id 3]
% *** IamStudent told you ***: Hello World.

If user 3 doesn't exist
[terminal of mine]
% tell 3 Hello World.
*** Error: user #3 does not exist yet. ***
%
```

- `yell <message>`:

Broadcast the message.

All the users(including yourself) will get the message with the following format:

```
*** <sender's name> yelled ***: <message>
```

Example:

```
Assume my name is 'IamStudent'
[terminal of mine]
% yell Good morning everyone.
*** IamStudent yelled ***: Good morning everyone.
%

[terminal of all other users]
% *** IamStudent yelled ***: Good morning everyone.
```

- `name <new name>`:

Change your name by this command. Broadcast the message with the following format:

```
*** User from <IP>:<port> is named '<new name>'. ***
```

Notice that the name CAN NOT be the same as other users' name, or you will get the following message:

```
*** User '<new name>' already exists. ***
```

Example:

```
[terminal of mine]
% name Mike
*** User from 140.113.215.62:1201 is named 'Mike'. ***
%

[terminal of all other users]
% *** User from 140.113.215.62:1201 is named 'Mike'. ***
```

If Mike is on-line, and I want to change name to Mike, this name change will fail.

```
[terminal of mine]
% name Mike
*** User 'Mike' already exists. ***
%
```

In all of the test cases, the maximum length of a user's name is **20** characters and will only consists of **alphabet and digits**.

3.3 Login/Logout message

- When a user login, broadcast as follows:

```
*** User '<user name>' entered from <IP>:<port>. ***
```

When a user logout, broadcast as follows:

```
*** User '<user name>' left. ***
```

Example:

```
[terminal of all users]
*** User '(no name)' entered from 140.113.215.63:1013. *** # user logins
*** User '(no name)' left. *** # user logouts
```

- Clients always use 'exit' to leave the shell.

3.4 Welcome Message

```
*****
** Welcome to the information server. **
*****
```

3.5 User Pipe

1. The formats of using user pipe are '(command) >n' and '(command) <n'. '>n' pipes result (**ONLY** stdout) of command into the pipe, '<n' reads contents from the pipe.
2. Display user pipe error message only to user who executes the command, not stderr.
3. Broadcast message when a user pipe is used. When a user writes into user pipe successfully. broadcast as follows:

```
*** <sender_name> (#<sender_id>) just piped '<command>' to <receiver_name> (#<receiver_id>) ***
```

Notice: Command should be whole command line

Example:

```
% cat <2 | number | number | cat
*** student1 (#1) just received from student2 (#2) by 'cat <2 | number | number | cat' ***
```

If the pipe already exists, show the following error message:

```
*** Error: the pipe #<sender_id>->#<receiver_id> already exists. ***
```

Whenever a user receives from the user pipe successfully, broadcast as follows:

```
*** <receiver_name> (#<receiver_id>) just received from <sender_name> (#<sender_id>) by '<command>' ***
```

If the pipe does not exist, show the following error message:

```
*** Error: the pipe #<sender_id>->#<receiver_id> does not exist yet. ***
```

If the sender or receiver does not exist, show the following error message:

```
*** Error: user #<user_id> does not exist yet. ***
```

Examples:

student1 (#1) pipes a command into student2(#2) via a pipe #1->#2.

```
<user1 login>
<user2 login>
% cat test.html >2
*** student1 (#1) just piped 'cat test.html >2' to student2 (#2) ***
% cat test.html >2
*** Error: the pipe #1->#2 already exists. ***
```

student2(#2) can receive from the pipe #1->#2.

```
% cat <1
*** student2 (#2) just received from student1 (#1) by 'cat <1' ***
...some output... # message from pipe #1->#2.
% cat <1
*** Error: the pipe #1->#2 does not exist yet. ***
% cat <3
*** Error: user #3 does not exist yet. ***
```

4. '>n' or '<n' has no space between them. So, you can distinct them from "> filename" easily.
5. The following situations will **NOT** appear in any test cases, so you don't need to worry about them: (1) output to/input from several pipes (e.g. user pipe, ordinary pipe, number pipe) or file.

```
% ls >2 | number
% ls >2 |1
% ls >2 > aa.txt
% ls >2 >4 // output to multiple number pipe

% cat test.html |1 // number pipe to next command
% cat <2 // input from number pipe and user pipe
% cat | number <1

% cat <2 >2 // user 2 pipe to himself/herself.
% cat <2 <1 // get multiple user pipe.
```

p.s. the following situations may happen:

```
% cat <2 | number
% cat <2 |1
% cat <2 > a.txt
% cat <2 >1
% cat >1 <2
% cat <2 >2
```

6. Show the message of '<n' first, then show the message of '>n'.

Example:

```
% cat <2 >1
*** student3 (#3) just received from student2 (#2) by 'cat <2 >1' ***
*** student3 (#3) just piped 'cat <2 >1' to student1 (#1) ***

% cat >1 <2
*** student3 (#3) just received from student2 (#2) by 'cat >1 <2' ***
*** student3 (#3) just piped 'cat >1 <2' to student1 (#1) ***
```

7. Show the error message of '<n' first, then show the error message of '>n' and unknown command.

Example:

```
% ctt <998 >999
*** Error: user #998 does not exist yet. ***
*** Error: user #999 does not exist yet. ***
```

```
Unknown command: [ctt].
```

```
% ctt >999 <998
*** Error: user #998 does not exist yet. ***
*** Error: user #999 does not exist yet. ***
Unknown command: [ctt].
```

p.s. the following situations may not happen:

```
% cat <998 | ctt | ls >999      // 'ctt' is an unknown command
```

8. Multiple client user pipe to one client.

```
(client 1)% ls >3
(client 2)% ls >3
(client 3)% cat <1
(client 3)% cat <2
```

3.6 Other requirements

1. Initial setting: You have to do this when new client connects.
 - Environment variables: Initial value of **PATH** is **bin:.**
Notice that every client will have its own environment variables settings.
Example:

```
[client A] PATH=.
[client B] PATH=bin:.
```
 - You can handle environment variables by yourself.
 - user name: default user name is (**no name**)
2. All behaviors required by project 1 are still required in this project for each user. All commands in project 1 should be working.

4 Specification

1. The maximum number of online users is 30. In other words, there will only be 30 users online simultaneously. However, there will be more than 30 users logged in in some test cases. For example: 30 users login -> 1 user logouts -> 1 user logins ...
2. The length of a single-line input will not exceed 15000 characters.
3. The length of each command will not exceed 256 characters.
4. The message of tell/yell will not exceed 1024 characters.
5. Any ' or ' in this specification must be replaced with '.
6. We only check client output, so you can print anything on your server.
7. To avoid race conditions, different clients don't execute commands at the same time.
8. Commands [who], [tell], [yell], [name] are single line commands, which means there will be no pipe connected to these commands, just like [printenv] or [setenv].

Example:

```
% ls | yell          // It is illegal. This will not appear in testcases.
% who |2             // It is illegal. This will not appear in testcases.
% printenv PATH > out.txt // It is illegal. This will not appear in testcases.
```

9. If you disconnect from the server, all user pipe about you (send or receive) should be closed.
10. You need to take care of the relative position between "% " and Broadcast message.
"% " will only sent when:

- (a) The client connected to it. In this situation, the order of the messages will be:

```
welcome message -> login Broadcast message -> "% "
```

Example:

```
*****
** Welcome to the information server. **
*****
*** User '(no name)' entered from 140.113.215.63:1013. ***
%
```

(b) Current command finished. (Except a line end with number pipe)

```
% ls
bin
test.html
%
```

Example:

After 4 users login, the message of user 1 will be:

```
*****
** Welcome to the information server. **
*****
*** User '(no name)' entered from 140.113.215.63:1013. ***
% *** User '(no name)' entered from 140.113.215.64:1014. ***
*** User '(no name)' entered from 140.113.215.65:1015. ***
*** User '(no name)' entered from 140.113.215.66:1016. ***
```

11. You can only implement this project with C and C++, other third-party libraries are NOT allowed.
12. You should set flag **SO_REUSEADDR** in server socket.
Hint: use function `setsockopt`
13. For the 1st server (single-process concurrent), you must use **select** to handle client's I/O.
14. For the 2nd server (concurrent connection-oriented), you must use **FIFO** to implement user pipe. The FIFO files should be put under directory "user_pipe"
15. For the 2nd server (concurrent connection-oriented), you must use **shared memory** to save client information and to broadcast or send message to clients.
16. For the 2nd server (concurrent connection-oriented), you must use **fork** to create your npshell clients.

5 Submission

- E3:

- (a) Create a directory named your **student ID**, put **ONLY** your source code files into the directory. **DO NOT** upload anything else (e.g. np_simple, np_single_proc, np_multi_proc, noop, removetag, test.html, .git, __MACOSX)
- (b) You must provide **Makefile**. Two executable files named **np_single_proc** (server 1), **np_multi_proc** (server 2) should be produced after typing **make** command in **top layer of the directory**.
- (c) The server should listen on the port assigned by the first argument.

Example:

```
./np_single_proc 12345 # Listen on port 12345
```

- (d) zip the directory and upload the .zip file to E3.

Attention !! we only accept .zip format

Example:

```
313540016
|-- Makefile
|-- np_single_proc.cpp # Server 1
|-- np_multi_proc.cpp # Server 2
|...
```

Zip the folder 313540016 into 313540016.zip and upload 313540016.zip to E3

- Github Classroom:
 - (a) Make sure you have accepted the invitation of Project 2
 - (b) You can push anything to Github Classroom, but make sure to commit **at least 10 times**.

6 Notes

1. We take plagiarism seriously. **You will get zero points on this project for plagiarism.**
2. You will lose points for violating any of the rules mentioned in this spec.
3. NP projects should be run on the NP server. Otherwise, your account may be locked.
4. Any abuse of NP server will be recorded.
5. Do not leave any zombie processes in the system.